PRACTICAL NO. 7

AIM: Stored Procedures and Exception Handling.

REQUIREMENTS:

An IDE (Integrated Development Environment) like MySQL, Oracle and etc.

THEORY:

1. **Stored Procedures:**

A stored procedure or function is a named PL/SQL module, which contain it. They can be compiled and stored in one of the Oracle engine's system tables. Stored Procedures and Functions are stored in the Oracle database. They are invoked or called by any the PL/SQL block that appears within an application. Before the procedure or function is stored, the Oracle engine parses and compiles the procedure or function. The Oracle engine compiles the PL/SQL block.

If an error occurs during the compilation of the procedure, an invalid procedure is created. The Oracle engine displays a message after creation that the procedure was created with compilation errors. It does not display the errors. These can be viewed using select statement or with show command as below:

SELECT * FROM user_errors;

Or

SHOW errors;

Parts of Procedures: Procedures are made up of:

- A declarative part
- An executable part
- An optional exception-handling part

a. **Declarative Part:** The declarative part is used to declare constants, variables, exceptions and subprograms. The objects are local to the procedure or function. The objects become invalid once the user exits from the procedure or function.
b. **Executable Part:** It is compulsory part, which is used to perform actions. Variables declared are put into this block. The executable part in PL/SQL block consisting of SQL and PL/SQL statements that assign values control execution and manipulate data. The data that is to be returned back to the calling environment is also returned from here.
c. **Exception Handling Part:** It is optional part to handle the errors raised during the execution of code in the executable part. We cannot transfer the flow or execution from the Exception Handling part to the Executable part.

We cannot impose the NOT NULL constraint on a parameter. Also, we cannot specify a constraint on the datatype.

Argument Modesin PL/SQL:

Argument modes are used to define the behaviour of formal parameters.

- **In:**It is default argument mode. It passes values to subprogram. It is a formal parameter acts like a constant. It is a formal parameter cannot be assigned a value. It can be constant, initialised variable, literal or expression.

- **Out:** It must be specified. It returns values to a caller. It is a formal parameter acts like an uninitialized variable and cannot be used in an expression and must assigned a value. It must be a variable.

Creating a Stored Procedure:

To create procedures and store them permanently in an Oracle database, we use the CREATE PROCEDURE statement, which we can use interactively from SQL * Plus.

Syntax:

CREATE OR REPLACE PROCEDURE procedure_name

IS

Declaration Section

BEGIN

Execution Section

EXCEPTION

Exception Section

END;

Description:

- **procedure_name:** It is the procedure name used to identify the procedures when it is called in a program.
- **Declaration:** It is optional declaration section where local variables, constant, exception or cursor declaration is placed.

Example:

```
Create or Replace Procedure CSEB
IS
BEGIN
dbms_output.put_line('GOOD CLASS.');
END;
```

Results Explain Describe Saved SQL History

```
Procedure created.

0.14 seconds
```

Fig-7.1 Creating a stored Procedure

Declaration, Creation, Calling a Procedure:

We can change the logical flow of statements within the PL/SQL block with the number of**control structures**.

There are following conditional control statements:

- If-Then-Statement
- If-Then-Else-Statement
- If-Then-Elseif-Statement (Ladder)

Below is an example where we will declare, create and call a procedure in a single code. In this we have declared three variables a, b and c with number as a data type. After that we have created a procedure in which we have declared 3 variables X, Y, and Z where in X and Y we will pass the values and at the end it will help to find the minimum value which will be stored in the variable Z.

Example:

```
DECLARE
  a number;
  b number;
  c number;
Procedure FindMin(X IN number, Y IN number, Z OUT number) IS
  BEGIN
    IF X < Y THEN
      Z := X;
    ELSE
      Z := Y;
    END IF;
  END;
BEGIN
  a := 28;
  b := 30;
  FindMin(a, b, c);
  dbms_output.put_line('Minimum value is ' || c);
END;
```

Results Explain Describe Saved SQL History

```
Minimum value is 28

Statement processed.
```

Fig-7.2 Finding minimum among two variables

PRACTICAL NO. 8

AIM: Triggers and Cursor Management in PL/SQL.

REQUIREMENTS:

An IDE (Integrated Development Environment) like Oracle.

THEORY:

1. Triggers in PL/SQL:

A database trigger is a stored procedure that is fired when an INSERT, UPDATE or DELETE statements is issued against the associate table. The name trigger is appropriate, as these are triggered(fired) whenever the mentioned commands are executed. A trigger defines an action the database should take when some database related event occurs.

A trigger is automatically executed without any action required by the user. A stored procedure on the other hand needs to be explicitly invoked.

Parts of a Trigger:

A database trigger has the following parts:

- Triggering event or statement
- Triggering Constraint(Optional)
- Trigger action

a. **Triggering Event or Statement:**It is the SQL statement that causes a trigger to be fired. A triggering event can be INSERT,UPDATE or DELETE statement for a specific table.

b. **Trigger Constraint or Restriction:** It specifies a Boolean(logical) expression that must be true for the trigger to fire. The trigger actions are not executed if the trigger restriction evaluates to FALSE. A trigger restriction is an option available for triggers that are fixed for each row. Its function is to conditionally control the action of the trigger.

c. **Trigger action:**A trigger action is a procedure(PL/SQL block) that contains the SQL statements and PL/SQL code to be executed when a triggering statement is issued and the trigger restriction evaluates to TRUE.

Triggers are written to be executed in response to DML, DDL and data operation. Triggers can be defined on table, view, schema or database with which an event is associated.

Syntax for Creating a Trigger:

CREATE OR REPLACE TRIGGER Trigger_name

{BEFORE|AFTER}

{| DELETE | [OR] INSERT | [OR] UPDATE [OF column_name, …]}

ON table_name

[REFERENCING {OLD AS o NEW AS n}]

[FOR EACH ROW [WHEN condition]]

DECLARE

Variable declaration;

Constant declaration;

BEGIN

PL/SQL subprogram body;

[EXCEPTION

Exception PL/SQL block;]

END;

Types of Triggers:

A trigger's type is defined by the type of triggering transaction and by the level at which the trigger is executed.

Oracle has following types of triggers depending on the different applications:

- Row Level Triggers
- Statement Level Triggers
- Before Triggers
- After Triggers

a. **Row Level Triggers:** They execute once for each row in a transaction. The commands of this triggers are executed on all rows that are affected by command that enables the trigger.

b. **Statement Level Triggers:** Statement Level triggers are triggered only once for each transaction. These are default types of triggers created by CREATE TRIGGER command.

c. **Before and After Trigger:** Since triggers are executed by events, they may be set to occur immediately before or after those events.
   - BEFORE triggers execute the trigger action before the triggering statement.
   - AFTER trigger executes the trigger action after the triggering statement is executed. These are used when you want the triggering statement to complete before executing the triggering action.

Examples:

a. Creating a trigger:

```
Autocommit  Display 10
CREATE OR REPLACE TRIGGER upper_trigger
BEFORE INSERT OR UPDATE OF employee_name ON employee
FOR EACH ROW

BEGIN
  :new.employee_name := UPPER(:new.employee_name);
END;

Results  Explain  Describe  Saved SQL  History

Trigger created.
```

Fig-8.1 Creating a Trigger(employee_name in uppercase)

b. Creating a trigger after updation:

```
Autocommit  Display 10
CREATE OR REPLACE TRIGGER employee_update
AFTER UPDATE OF employee_name ON employee FOR EACH ROW
BEGIN
  dbms_output.put_line('OLD NAME:' || :OLD.employee_name);
  dbms_output.put_line('NEW NAME:' || :NEW.employee_name);
END;

Results  Explain  Describe  Saved SQL  History

Trigger created.
```

Fig-8.2 Creating a Trigger (after an updation on employee_name)

c. **Dropping a Trigger:**

Triggers may be dropped via the drop rigger command. In order to drop a trigger, one must either own the trigger or have DROP ANY TRIGGER system privilege.

Syntax:

DROP TRIGGER trigger_name;

Example:

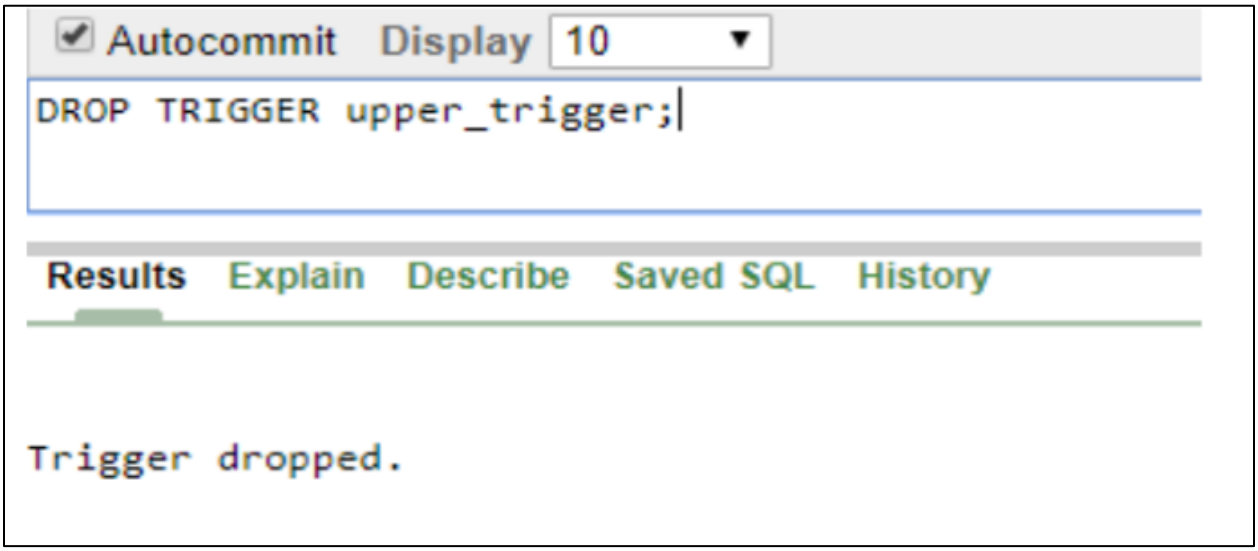

Fig-8.3 Trigger Dropped

2. Cursor Management in PL/SQL:

A cursor is a temporary work area created in system area when SQL statement is executed. It contains information on select statement and row of data access by it. The Oracle Engine uses a work area to execute SQL area to execute SQL statements and store information. A cursor is a PL/SQL construct that allows us to name these work areas, ad to access their stored information. The data stored in the cursor is called Active Data Set.

Types of Cursors:

There are two types of cursors:

- Implicit Cursors
- Explicit Cursors

i) Implicit Cursors:

Implicit cursors are created by default when DML statement is executed. Implicit cursors are declared by PL/SQL implicitly for all SQL statements. They are opened and managed by Oracle engine internally. So there is no need to open and manage by the uses, these are operations that are performed automatically.

ii) Explicit Cursors:

Explicit cursor is created by user when executing select statement. Explicit cursors are user- defined cursors for processing of multiple records returned by a query. Explicit cursors are declared explicitly, along with other identifiers actions. These are defined in the DECLARE section of PL/SQL block.

General Cursor Attributes:

Whenever any cursor is opened and used, the oracle engine creates a set of four system variables, which keeps track of the current status of the cursor. These variables can be accessed and used in PL/SQL block. Both implicit and explicit cursor has four attributes.

The attributes are described as:

%IS OPEN: It returns TRUE if cursor is open, FALSE otherwise.

%FOUND: It returns TRUE is record was fetched successfully from the opened cursor, and FALSE otherwise.

%NOTFOUND: It returns TRUE if record was not fetched successfully and FALSE otherwise.

%ROWCOUNT: It returns number of records processed from cursor.

Example:

Autocommit Display 10

```
DECLARE
   total_rows number(2);
BEGIN
   UPDATE employee
   SET salary = salary + 500;
   IF sql%notfound THEN
      dbms_output.put_line('no employee selected');
   ELSIF sql%found THEN
      total_rows := sql%rowcount;
      dbms_output.put_line( total_rows || ' employee selected ');
   END IF;
END;
```

Results Explain Describe Saved SQL History

```
6 employee selected

1 row(s) updated.
```

Fig-8.4 Cursor Management